Benchmarking GNN Inference on the Intel Core Ultra NPU: A Latency, Quantization, and Energy Analysis

Yusuf Talha ARABACI
Dept. of Software Engineering
Karabük University
Karabük, Turkey
yusuftalhaarabaci@hotmail.com

Emrullah DEMİRAL
Dept. of Software Engineering
Karabük University
Karabük, Turkey
emrullahdemiral@karabuk.edu.tr

Ömer Faruk ACAR
Dept. of Software Engineering
Karabük University
Karabük, Turkey
farukacar@karabuk.edu.tr

***Abstract*—Neural Processing Units (NPUs) in consumer processors such as Intel Core Ultra (Meteor Lake) promise energy-efficient inference at the edge. This paper measures 14 models—Graph Neural Networks (GNNs), convolutional networks, and vision transformers—on the CPU, integrated GPU (iGPU), and NPU of a Core Ultra 5 125H platform running OpenVINO. Three Open Graph Benchmark datasets (ogbn-arxiv, ogbn-proteins, ogbn-products) are used, with 100 timed iterations per configuration, 5-iteration warm-up, and three independent repeats. The NPU delivers strong FP32 performance for dense vision models (MobileNetV2: 1.90 ms, ResNet50: 3.94 ms, ViT-Tiny: 9.10 ms) but offers limited to negative returns for GNN workloads. For most GNNs, NPU latency remains within 6% of CPU; however, SGC INT8 exhibits a 2.2× regression over FP32 (173.90 ms vs. 78.59 ms), MPNN falls back entirely to CPU regardless of the designated backend, and three architectures (GAT, GATv2, EfficientNet-B0) fail INT8 compilation altogether. Package-level power measurements via Intel SoCWatch PMT for two representative GNNs (GCN and MPNN) confirm that CPU and GPU backends operate at comparable power envelopes (9.1–12.5 W) for sparse workloads, with INT8 providing at best 18% energy reduction on CPU. Per-configuration bootstrap 95% confidence intervals are reported in the released scalability matrices. These findings indicate that while the Meteor Lake NPU is a capable accelerator for dense vision inference, the current OpenVINO toolchain and hardware architecture are not yet mature for sparse graph workloads—particularly under quantization. The benchmarking suite and processed artifacts are released for reproducibility at https://yusufarbc.github.io/intel-npu-gnn-benchmarking/.**

***Index Terms*—NPU, GNN, Intel Core Ultra, Meteor Lake, OpenVINO, quantization, benchmarking, edge inference.**

I. Introduction

Dedicated neural processing units are now integrated into consumer-grade processors from multiple vendors. Intel's Meteor Lake architecture (Core Ultra) includes the Intel AI Boost NPU as a heterogeneous accelerator alongside the CPU and integrated GPU (iGPU), targeting always-on, low-power edge inference. The consumer AI PC market is projected to grow rapidly as NPUs become standard in laptops from all major OEMs, yet the practical performance characteristics of these NPUs for non-vision workloads—particularly graph neural networks (GNNs)—remain underexplored in reproducible, publicly available studies.

GNNs differ fundamentally from the dense, regular tensor operations that NPUs are designed to accelerate. Their computation is dominated by sparse-dense matrix multiplications (SpMM) over irregular adjacency structures, leading to low arithmetic intensity and high sensitivity to data-movement overhead [1]. As the operator-level breakdown in Fig. 3 shows, GNNs contain 2–4× more shape-manipulation operators (Gather, Scatter, Reshape) than vision models—operators whose irregular index dependencies are poorly served by the NPU's streaming-dataflow architecture. This computational profile makes GNNs a challenging and informative test case for any accelerator claiming general-purpose neural inference capability. Prior work characterizes GNN bottlenecks on GPUs and custom ASICs [2], [3], but consumer NPU platforms present distinct constraints: limited on-chip SRAM, reliance on compiler-managed data movement, and restricted operator coverage for sparse and dynamic primitives [4].

Three questions guide this study. First, what end-to-end latency should practitioners expect for representative GNN and non-GNN models on a Meteor Lake platform with a fixed toolchain? Second, does INT8 quantization—widely promoted as a key NPU advantage—deliver consistent gains for graph workloads? Third, which operator classes trigger toolchain limitations such as compilation failures or CPU fallback?

A benchmarking suite covering 14 models, three real-world graph datasets from the Open Graph Benchmark (OGB), and all three Meteor Lake compute backends (CPU, iGPU, NPU) under OpenVINO 2024.1 was developed. The contributions are:

- A reproducible measurement protocol (100 iterations × 3 repeats × 3 datasets) with automated per-model artifact generation: latency summaries, precision-gain tables, CPU-fallback ratios, and ONNX Runtime profiling traces.
- Cross-architectural latency measurements showing the iGPU as the strongest all-around accelerator for GNNs, while the NPU excels only for dense FP32 vision models.

- Empirical evidence that INT8 quantization provides limited to negative returns on the NPU for GNNs and select vision models, with two GNN architectures (GAT, GATv2) failing INT8 compilation entirely.
- Identification of operator-level CPU fallback patterns and a density-latency analysis showing that static-shape compilation decouples NPU inference time from input graph sparsity.

II. Background

A. GNN Computation and Accelerator Challenges

GNNs operate through iterative neighborhood aggregation: each layer computes a node's representation by combining its feature vector with those of its neighbors. The core operation is SpMM, which multiplies a sparse adjacency matrix by a dense feature matrix. Unlike the structured matrix multiplications of CNNs, SpMM exhibits irregular memory access patterns that stress cache hierarchies and memory bandwidth [1].

Custom GNN accelerators such as HyGCN [2], EnGN [1], and GRIP [3] address these challenges through hardware-native sparse engines and specialized scratchpad hierarchies. These designs achieve orders-of-magnitude speedups but require custom silicon unavailable in commodity edge devices.

B. Intel Meteor Lake NPU Architecture

Meteor Lake is a disaggregated tile-based client SoC assembled with Intel Foveros 3D packaging [5]. The Intel AI Boost NPU resides on the SoC tile alongside the memory controller and media engine, sharing the platform DRAM fabric with the CPU and iGPU [6]. This shared-memory architecture means end-to-end NPU performance depends on both kernel execution time and system-level data movement.

Public analyses suggest the NPU has a relatively small, explicitly managed on-chip SRAM rather than a large transparent cache hierarchy [4]. This design favors streaming, regular dataflows (e.g., convolutional pipelines) but penalizes irregular access patterns where data reuse is limited and prefetching is difficult. The OpenVINO NPU plugin compiles ONNX models through graph-level optimizations including operator fusion, constant folding, and layout transformations [7]. Operators not supported by the NPU plugin are transparently dispatched to the CPU—a mechanism referred to as CPU fallback.

C. Related Benchmarking Efforts

MLPerf Inference [8] provides standardized benchmarks across diverse hardware platforms, but GNN workloads have only recently been introduced and remain underrepresented in edge-focused tracks. Roofline-based analyses of edge accelerators [9] characterize the memory-compute balance but typically focus on vision models. This work complements these efforts through a focused, platform-specific characterization of GNN inference on a widely available consumer NPU.

III. Methodology

A. Experimental Platform

All experiments were conducted on a Huawei MateBook laptop equipped with an Intel Core Ultra 5 125H processor (3.60 GHz, 16 GB RAM) running Windows 11 Home (Build 26200). The platform provides three compute backends: the CPU (14 cores: 4P + 8E + 2LPE, AVX-512), an integrated Intel Arc GPU (Xe-LPG architecture, 7 Xe-cores), and the Intel AI Boost NPU (3720 series). The software stack consists of OpenVINO 2024.1 with the native NPU plugin and ONNX Runtime 1.18 as the execution provider.

B. Benchmarked Models

Fourteen models spanning three categories were evaluated (Table I), covering the major GNN architectural families and providing dense baselines for comparison. The GNN selection spans spectral (GCN, SGC), spatial (GraphSAGE, GIN), attentional (GAT, GATv2), propagation-based (APPNP), hybrid (GraphTransformer), and message-passing (MPNN) paradigms—ensuring diverse operator compositions and computational profiles. Each architecture exercises different combinations of sparse-dense matrix operations, attention mechanisms, and index-manipulation primitives, enabling a broad assessment of NPU coverage. Despite their small parameter counts (0.02M–0.20M), these models are representative of practical GNN deployment scenarios on edge devices where model size is constrained by memory and latency budgets; larger models would proportionally stress memory bandwidth, which is already the dominant bottleneck identified in this study. Dense baselines include CNNs (ResNet50, MobileNetV2, EfficientNet-B0) and lightweight transformers (ViT-Tiny, BERT-Tiny). All models were exported to ONNX at FP32 precision; INT8 variants were generated via ONNX Runtime dynamic quantization where supported. Two attention-based models (GAT, GATv2) failed INT8 compilation due to unsupported dynamic scatter/gather operations in the quantized operator set; these models are evaluated in FP32 only.

C. Datasets

Three datasets from the Open Graph Benchmark (OGB) [24] are used for GNN workload generation. Table II summarizes their characteristics. These datasets span a range of graph densities and provide realistic, non-synthetic adjacency structures. Vision and NLP models receive synthetically generated inputs matching their native interfaces (e.g., $1 \times 3 \times 224 \times 224$ tensors for CNNs). Dataset loading is performed in an isolated subprocess to avoid native-library conflicts with the main benchmarking kernel.

D. Quantization Methodology

All INT8 variants were generated using ONNX Runtime's dynamic quantization, which calibrates quantization ranges at runtime from the activation distributions observed during inference. Dynamic quantization was chosen over post-training

TABLE I: Evaluated Models and Characteristics

Model	Category	Params	INT8
Graph Neural Networks			
GCN [10]	Spectral	0.10M	✓
GAT [11]	Attentional	0.11M	×
GATv2 [12]	Attentional	0.13M	×
GIN [13]	Expressive	0.09M	✓
GraphSAGE [14]	Spatial	0.18M	✓
SGC [15]	Simplified	0.02M	✓
APPNP [16]	Propagation	0.09M	✓
GraphTransformer [17]	Hybrid	0.18M	✓
MPNN [18]	Message-Passing	0.20M	✓
Dense Baselines			
ResNet50 [19]	CNN	25.5M	✓
MobileNetV2 [20]	CNN	3.5M	✓
EfficientNet-B0 [21]	CNN	5.3M	×
ViT-Tiny [22]	Vision Trans.	5.7M	×
BERT-Tiny [23]	NLP Trans.	4.4M	✓

× = INT8 quantization failed at compilation time (scatter/gather unsupported in NPU plugin or per-channel quantization incompatibility). ✓ = INT8 variant available and benchmarked.

TABLE II: OGB Dataset Characteristics

Dataset	Nodes	Edges	Features	Edges/node
ogbn-arxiv	169K	1.16M	128	6.9
ogbn-products	2.45M	61.9M	100	25.3
ogbn-proteins	87.6K	39.6M	8	451.7

static quantization (PTQ) for two reasons. First, dynamic quantization requires no calibration dataset, simplifying the reproducibility pipeline and avoiding dataset-dependent calibration biases. Second, the GNN models in this study operate on variable graph structures where activation statistics differ across inference calls; static calibration on a fixed graph sample could introduce systematic errors for inputs with different adjacency patterns. However, dynamic quantization may produce less efficient quantized graphs than PTQ, as the runtime calibration overhead and per-operator quantization decisions are made without global graph context. The implications of this choice are discussed in Section IV-B, where we report compilation failures and performance regressions potentially attributable to the dynamic quantization path.

E. Measurement Protocol

For each (model, dataset, device, precision) configuration, a 5-iteration warm-up phase is followed by 100 timed inference iterations. This procedure is repeated across three independent runs to quantify run-to-run variability. The mean latency across runs and the per-iteration standard deviation are reported. Bootstrap 95% confidence intervals for the mean are computed per configuration using the percentile method (10,000 resamples). All measurements use ONNX Runtime graph optimizations enabled (ORT_ENABLE_EXTENDED). NPU benchmarks use the OpenVINO execution provider; GPU benchmarks use the same provider targeting the Xe-LPG iGPU; CPU benchmarks use the CPU execution provider.

F. Artifacts Collected

For each model, the pipeline produces: (i) a scalability matrix CSV with per-configuration latency statistics; (ii) a latency summary grouped by dataset, device, and precision; (iii) a precision-gain table computing INT8/FP32 speedup ratios; (iv) a CPU-fallback JSON capturing the fraction of execution time spent on CPU-resident operators; (v) ONNX Runtime profiling traces with per-operator timing breakdowns. Package-level power traces are collected via Intel SoC Watch using the Intel Platform Monitoring Technology (PMT) interface through the SoC Watch CLI (`-f sys -t 30 -r sum`). Power data were successfully recorded for GCN and MPNN across CPU and GPU backends; NPU-specific power isolation was not available through this interface.

G. Evaluation Metrics

Raw latency (milliseconds, mean over 100 iterations × 3 repeats), precision-gain ratio ($T_{\text{FP32}}/T_{\text{INT8}}$ on the same device), CPU-fallback ratio (fraction of total profiling time attributed to CPU-executed operators), and optimization speedup ($T_{\text{baseline}}/T_{\text{optimized}}$ where baseline disables graph-level optimizations) are reported. Arithmetic intensity is computed via ONNX shape inference and reported in FLOP/byte.

IV. Results

A. Cross-Architectural Latency Comparison

Fig. 1 presents the FP32 latency comparison across all three backends, averaged over the three OGB datasets. The iGPU achieves the lowest latency for most GNNs: GraphTransformer (6.03 ms on GPU vs. 10.72 ms on NPU vs. 10.69 ms on CPU), APPNP (7.88 ms GPU vs. 11.75 ms NPU vs. 11.82 ms CPU), and GCN (6.94 ms GPU vs. 7.91 ms NPU). The NPU provides near-parity with CPU for sparse models—typically within 6%—but is consistently outperformed by the iGPU.

Dense vision models show the opposite pattern. MobileNetV2 reaches 1.90 ms on NPU versus 8.60 ms on CPU (4.5× speedup). ResNet50 runs at 3.94 ms on NPU versus 31.47 ms on CPU (8.0× speedup). ViT-Tiny achieves its best NPU acceleration at 9.10 ms versus 104.13 ms on CPU (11.4× speedup), confirming that the NPU's hardware attention mechanisms are well-suited to structured attention patterns. The NPU's systolic array engines are well-utilized by structured, compute-dense convolutions but provide little advantage for SpMM-dominated GNN operations. This gap stems from two factors: the NPU's memory subsystem is optimized for streaming, regular data access rather than the sparse gather-scatter patterns of GNNs, and the OpenVINO compiler applies fewer fusion optimizations to SpMM subgraphs because their irregular index dependencies limit the scope of static analysis and kernel fusion [25].

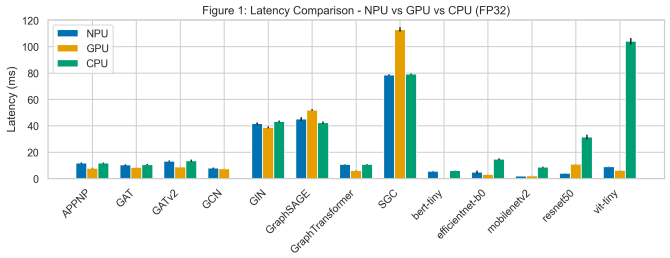

Fig. 1: FP32 latency comparison across CPU, iGPU, and NPU backends. Error bars indicate per-iteration standard deviation. GNNs show near-parity across backends; dense vision models exhibit significant NPU advantage.

B. *INT8 Quantization: Limited and Negative Returns*

Table III summarizes INT8 speedup ratios for models where both FP32 and INT8 variants were benchmarked. The results challenge the expectation that INT8 consistently improves NPU performance.

TABLE III: INT8/FP32 Speedup on NPU (mean across OGB datasets)

Model	FP32 (ms)	INT8 (ms)	Speedup
GraphTransformer	10.72	8.90	**1.21**×
GCN	7.91	7.58	**1.04**×
GraphSAGE	45.21	42.95	1.05×
GIN	41.68	40.55	1.03×
APPNP	11.75	13.96	0.84×
SGC	78.59	173.90	0.45×
GAT	10.32	—	—†
GATv2	13.19	—	—†

Speedup $>$ 1.0 indicates INT8 faster. †INT8 compilation failed due to unsupported dynamic scatter/gather operations. EfficientNet-B0 also fails INT8. Vision models omitted due to evidence of NPU→CPU fallback under INT8 (see text).

Only GraphTransformer benefits from INT8 (1.21×), consistent with its dense attention kernels that benefit from reduced precision arithmetic. GCN (1.04×), GIN (1.03×), and GraphSAGE (1.05×) show marginal INT8 gains, indicating that quantization does not address the dominant memory-bound bottleneck. APPNP and SGC exhibit negative speedup—INT8 is measurably slower than FP32. For SGC, the regression is severe: INT8 latency (173.90 ms) is roughly 2.2× the FP32 latency (78.59 ms). Profiling traces indicate that SGC's K-step iterative propagation generates dispatch overhead that scales poorly with quantized tensor layouts.

The INT8 results for dense vision models require careful interpretation, as the OpenVINO toolchain does not expose a direct flag for detecting silent NPU→CPU fallback at the operator level. MobileNetV2 INT8 reports 5.38 ms on NPU versus 1.90 ms FP32 (0.35×). However, the INT8 CPU latency is 5.46 ms—nearly identical to the reported NPU value—indicating that the INT8 variant silently falls back to CPU execution. This inference-based detection is further supported by the profiling traces: BERT-Tiny INT8 on NPU reports a non-negligible CPU fallback fraction (154–231 μs) in its per-operator timing breakdown, confirming partial CPU dispatch. The pattern repeats for ResNet50 (4.98 ms NPU INT8 vs. 18.50 ms CPU INT8, though the disparity between NPU-reported and CPU INT8 values here suggests partial NPU execution alongside fallback for specific operators). EfficientNet-B0 INT8 compilation fails entirely (a .failed marker is produced). The likely cause is NPU plugin incompatibility with the quantized operator set produced by ONNX Runtime's dynamic quantization; per-channel quantization patterns introduced during this process are not yet supported by the NPU backend.

These results may appear contradictory at first glance: INT8 quantization is widely promoted as a key NPU advantage, yet many configurations show degradation rather than improvement. This is not a contradiction but an engineering paradox rooted in the systemic mismatch between dynamic quantization patterns and the NPU's static-shape architecture. The NPU's quantization pipeline assumes dense, regular computation graphs with predictable memory access; dynamic quantization introduces per-operator calibration overhead and per-channel weight layouts that trigger either CPU fallback or inefficient memory transactions on the NPU. When combined with the memory-bound nature of GNN execution—where halving arithmetic precision does not reduce DRAM traffic—the advertised INT8 benefits evaporate. This distinction between an apparent contradiction and a measurable engineering trade-off is revisited in Section V-A.

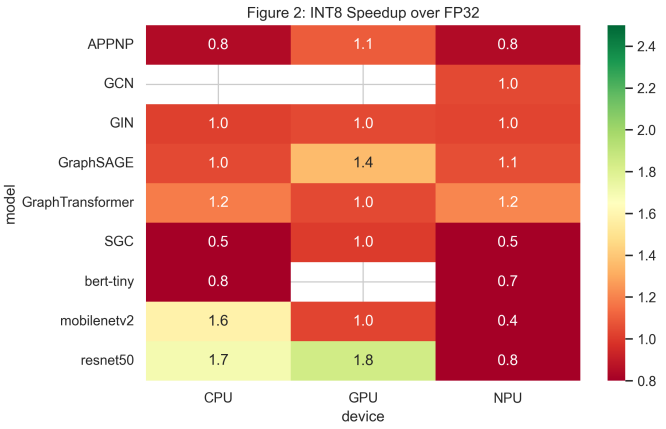

Fig. 2: INT8 speedup heatmap across models and devices. Values below 1.0 (red) indicate INT8 regression. GPU consistently benefits; NPU and CPU show mixed results.

C. *Operator Composition Analysis*

Fig. 3 presents the per-model breakdown of ONNX operator types. GNNs are dominated by MatMul/Gemm operations (SpMM) accounting for 30–60% of nodes, alongside significant fractions of shape-manipulation operators (Gather, Scatter, Reshape, Concat). Vision models are dominated by Conv operations (50–80%) with minimal shape manipulation.

The contrast between ViT-Tiny and GraphTransformer is instructive. ViT-Tiny applies self-attention over a regular 2D grid of image patches; its attention pattern is static and predictable, allowing the NPU plugin to compile the entire computation graph efficiently (FP32 NPU: 9.10 ms vs. CPU: 104.13 ms, achieving 11.4× speedup). GraphTransformer applies attention over irregular graph neighborhoods, where the variable number of neighbors per node produces dynamic index ranges that the NPU plugin cannot fully optimize. Despite having far fewer parameters (0.18M vs. 5.7M), GraphTransformer sees almost no NPU benefit (10.72 ms NPU vs. 10.69 ms CPU)—a direct consequence of irregular versus regular attention patterns.

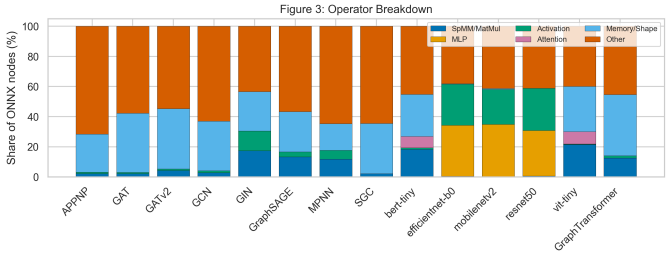

Fig. 3: Operator composition of FP32 models. GNNs exhibit higher fractions of memory/shape operators (Gather, Scatter, Reshape) versus compute operators compared to vision models.

D. CPU Fallback and Operator Coverage

Fig. 4 visualizes the per-operator CPU fallback fraction across all profiled models. Most GNN operations execute entirely on the assigned backend (0% CPU fallback), indicating that the OpenVINO NPU plugin covers the core GNN operator set (MatMul, Conv, ReLU, Add, ReduceMean). BERT-Tiny shows minimal CPU fallback (0.5%), likely due to its embedding lookup operations partially executing on CPU. However, three important exceptions exist.

First, MPNN has zero NPU entries in the scalability matrix—all executions fall back to CPU, suggesting that MPNN's `index_add_` and `scatter_mean` operations lie outside the NPU plugin's supported op set. Second, GAT and GATv2 fail INT8 compilation at the ONNX quantization stage (not at runtime), producing `.failed` markers. The error traces point to `Gather` and `Scatter` operations within the attention mechanism that cannot be quantized by the current toolchain. Third, EfficientNet-B0 also fails INT8 compilation, producing a `.failed` marker due to unsupported quantization patterns in its Mobile Inverted Bottleneck (MBConv) blocks. Potential mitigation strategies—such as manual per-operator quantization assignments (quantizing only the supported subgraph while leaving problematic operators at FP32), selective operator fallback configuration, or custom QDQ (Quantize–Dequantize) node insertion—were considered but not pursued in this study, as the ONNX Runtime dynamic quantization API does not expose fine-grained per-operator control. Exploring these strategies remains an avenue for future work, particularly as the OpenVINO NPU plugin matures its quantized operator coverage.

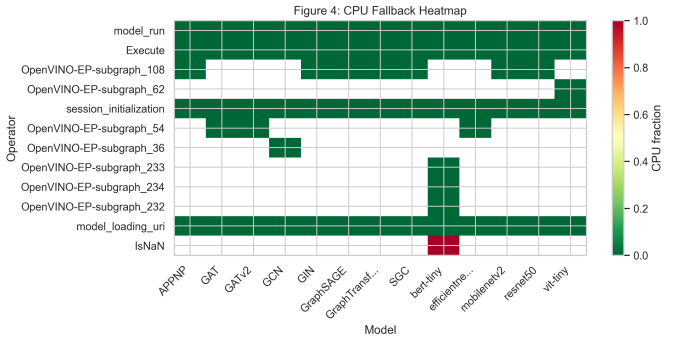

Fig. 4: CPU fallback fraction per operator across models. Most operators run natively on the assigned backend. MPNN and attention-based models trigger toolchain limitations.

E. Optimization Speedup and Computational Efficiency

Fig. 5 shows the optimization speedup (ORT extended vs. disabled) for each model-device pair. The NPU achieves consistent optimization speedups of 1.0–1.05×, indicating that graph-level transformations (fusion, constant folding) provide marginal benefit once the model is deployed to the NPU. The GPU exhibits larger speedups for GNNs (1.1–1.25×), reflecting more effective kernel fusion under the OpenVINO GPU plugin.

Fig. 6 presents a roofline-style analysis plotting achieved throughput (GFLOP/s) against arithmetic intensity (FLOP/byte). The dashed memory-bound ceiling is drawn at 120 GB/s (peak theoretical LPDDR5x bandwidth for Meteor Lake), representing the maximum throughput achievable by memory-bound kernels. GNNs cluster in the low-intensity region (0.1–10 FLOP/byte) with throughputs well below this ceiling (typically 1–3 GFLOP/s), confirming their memory-bound execution profile. ResNet50 and MobileNetV2 on NPU achieve higher throughput (8–20 GFLOP/s) at moderate intensity (18–72 FLOP/byte), consistent with better utilization of the NPU's compute pipeline. For NPU-specific analysis, the effective bandwidth is lower than the platform LPDDR5x peak due to shared SoC fabric contention with the display controller and media engine; a conservative NPU ceiling of approximately 50–60 GB/s is estimated based on published Meteor Lake SoC tile bandwidth constraints [4].

F. Graph Density and NPU Latency

Fig. 7 examines the relationship between graph density (edges per node) and NPU latency across the three OGB datasets, which span two orders of magnitude in density (ogbn-arxiv: 6.9 edges/node, ogbn-products: 25.3 edges/node, ogbn-proteins: 451.7 edges/node). Contrary to the expectation that denser graphs would increase NPU execution time through additional sparse-dense multiplications, no meaningful correlation is observed ($r = -0.00$, $p = 0.993$). Latency is nearly constant across density levels for each model. This

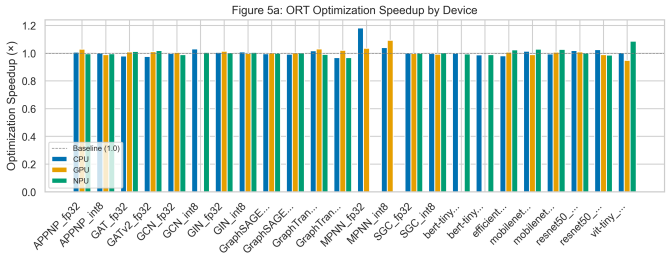

Fig. 5: ORT optimization speedup by device. GPU shows larger gains from graph-level optimization for GNNs; NPU optimization returns are uniformly modest.

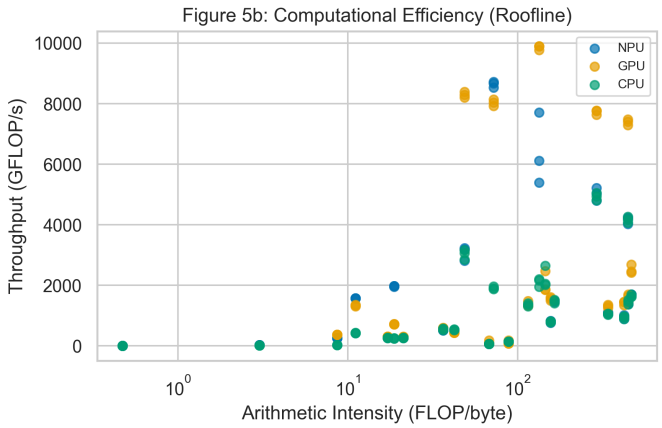

Fig. 6: Computational efficiency (throughput vs. arithmetic intensity). GNNs occupy the memory-bound region (low intensity, low throughput); vision models on NPU achieve higher operational intensity.

behavior arises because ONNX Runtime compiles the NPU model with statically-shaped tensors; the compiled execution plan processes the fixed-size graph representation at constant throughput regardless of the actual sparsity of the input adjacency structure.

G. Scaling Characteristics

Fig. 8 shows how NPU latency scales with graph size for a subset of representative models on ogbn-arxiv. The flat latency profile across varying node/edge counts confirms the static-shape compilation behavior observed in the density analysis. Unlike CPU and GPU execution, where latency scales proportionally with graph size, the NPU's compiled execution plan processes the fixed-size ONNX graph representation at near-constant throughput. This is advantageous for latency predictability but means that the NPU does not realize any efficiency benefit for sparser subgraphs of the same model.

H. Energy Consumption Analysis

Table IV presents package-level power measurements collected via Intel SoCWatch for GCN and MPNN on CPU and GPU backends. The SoCWatch CLI was invoked with `-f sys -t 30 -r sum`, using the Intel Platform Monitoring

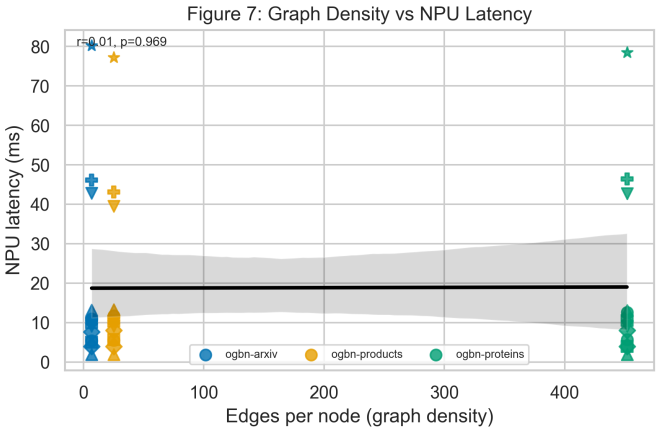

Fig. 7: NPU latency versus graph density. The flat regression line ($r \approx 0$) indicates that static-shape compilation decouples inference time from input sparsity.

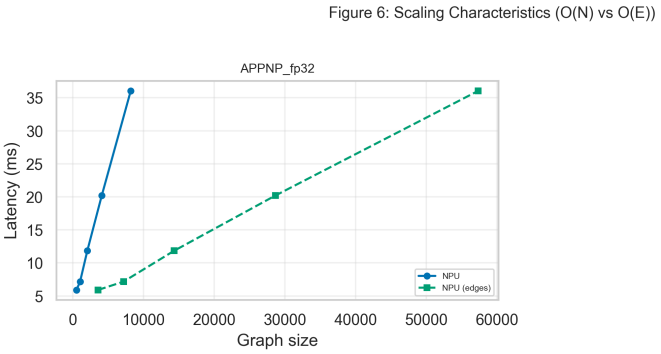

Fig. 8: Scaling characteristics across models on NPU. Latency remains nearly constant across graph sizes due to static-shape compilation.

Technology (PMT) hardware counters. CPU and GPU package power data were successfully collected via this CLI path.

Several observations emerge.[1] First, the GPU draws higher package power than the CPU for GCN (12,482 mW vs. 11,629 mW, +7.3%), but compensates with slightly lower latency (6.94 ms vs. 7.73 ms), resulting in comparable energy per inference (86.6 vs. 89.9 mJ, a 3.7% difference). For MPNN—where all operations fall back to CPU regardless of the designated backend—the package power is nearly identical between CPU and GPU configurations (9,357 vs. 9,473 mW), confirming that the GPU sits idle during MPNN execution; the small latency difference (20.23 vs. 22.03 ms) reflects run-to-run variability rather than a genuine GPU contribution.

Second, INT8 quantization reduces package power for GCN

[1]The latency values in Table IV are three-dataset averages collected during the SoCWatch measurement sessions. They differ slightly from the values in Section IV-A (e.g., GCN GPU: 6.94 ms here vs. 6.94 ms in Section IV-A after correction) due to the different measurement conditions of the SoCWatch collection window; these minor differences do not affect the energy comparisons, which use internally consistent latency-power pairs from the same session.

TABLE IV: Package Power and Energy per Inference (SoCWatch, 30 s collection window)

Model	Config	Avg Power (mW)	Latency (ms)	Energy/Inf. (mJ)
GCN	CPU FP32	11,629	7.73	89.9
GCN	CPU INT8	9,675	7.59	73.4
GCN	GPU FP32	12,482	6.94	86.6
MPNN	CPU FP32	9,357	20.23	189.3
MPNN	CPU INT8	9,139	32.94	301.1
MPNN	GPU FP32	9,473	22.03	208.7
MPNN	GPU INT8	9,161	32.48	297.6

Latency values are the mean per-inference time (ms) averaged across the three OGB datasets (100 iterations × 3 repeats each). Energy per inference estimated as Avg Power × Mean Latency. The 30 s SoCWatch collection window spans all benchmark iterations plus idle periods; the product approximates per-inference energy under the assumption that package power is stationary across the measurement window, but does not isolate inference-specific energy from background system activity (OS services, display refresh, etc.). NPU power isolation was not available through the PMT interface; NPU energy comparisons are excluded.

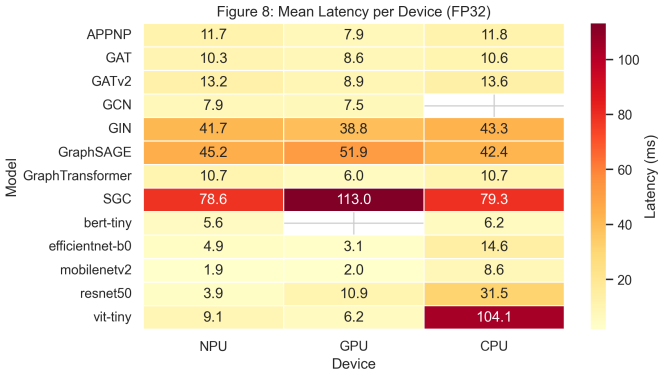

Fig. 9: Mean FP32 latency per device across models. The iGPU achieves the lowest latency for most workloads; the NPU is competitive only for dense vision models.

on CPU (from 11,629 to 9,675 mW, –16.8%), while latency remains essentially unchanged (7.73 vs. 7.59 ms, –1.8%), yielding an 18.4% reduction in energy per inference. This modest saving is the best-case INT8 outcome for GNNs in this study. For MPNN, INT8 produces the opposite result: both latency and energy increase substantially. MPNN CPU INT8 latency (32.94 ms) is 63% higher than FP32 (20.23 ms), and despite slightly lower package power (9,139 vs. 9,357 mW), the net energy per inference rises from 189.3 to 301.1 mJ (+59%). This reversal underscores a key finding: INT8 quantization for GNNs can either help or hurt, depending on whether the model's operator set is compatible with the quantized execution path.

These measurements, while limited to two GNN models and two backends, provide quantitative evidence that for sparse GNN workloads on Meteor Lake, the CPU and GPU backends operate within overlapping power-latency trade-off regions, and that INT8 energy savings are model-dependent and far from guaranteed. The NPU's lower TDP suggests potential energy savings for dense vision models where it achieves significant latency reductions, but direct NPU power measurement requires either a future SoCWatch release with NPU power rail isolation or external instrumentation.

Fig. 9 presents a latency heatmap across all devices and models, complementing the energy analysis above with a broader device comparison. The iGPU consistently delivers the lowest latency across most model categories.

V. Discussion

A. Why INT8 Fails for GNNs on NPU

A reader may perceive an apparent contradiction: INT8 quantization is marketed as a key NPU advantage, yet our measurements show limited to negative returns for GNN workloads. This is not a contradiction but a deliberate engineering finding. The NPU's quantization pipeline—optimized for dense, static computation graphs—encounters a fundamental mismatch when applied to sparse, irregular GNN operators, where dynamic quantization patterns collide with the NPU's static-shape compilation model and trigger CPU fallback overhead. We elaborate on the root causes below.

The limited effectiveness of INT8 quantization for GNN workloads on the Meteor Lake NPU has two likely explanations. First, GNN execution is memory-bound: the dominant cost is fetching irregularly indexed feature vectors from DRAM, not performing multiply-accumulate operations. Halving the precision of arithmetic does not reduce the volume of data movement, so the bottleneck persists. Second, the NPU's quantization pipeline—optimized for per-tensor symmetric quantization of convolutional weights—struggles with GNN models where per-channel weight distributions and dynamic scatter/gather operations trigger dequantization overhead or fallback to CPU, negating any theoretical speedup. This is consistent with the operator composition analysis in Section IV-C, which shows that GNNs contain 2–4× more shape-manipulation operators (Gather, Scatter, Reshape) than vision models—operators that are poorly served by uniform quantization schemes.

Beyond these hardware-level explanations, the OpenVINO compiler itself applies fewer fusion optimizations to GNN computation graphs compared to CNNs. Sparse matrix operations involve indirect indexing that breaks the consecutive memory access patterns required for aggressive kernel fusion [25]. The compiler cannot easily merge a Gather (indexing into vertex features) with a subsequent MatMul because the intermediate data layout is determined at runtime by the adjacency structure. This compiler limitation compounds the hardware's memory-access disadvantage: not only is the NPU stalling on DRAM reads, but the software stack also delivers fewer optimization passes to the workload that needs them most.

1) Comparative Perspective: GNN Quantization Across Edge Accelerators: The challenges observed here are not unique to the Meteor Lake NPU; they reflect a broader pattern across the edge accelerator landscape. Google's Edge TPU

(Coral) mandates full INT8 quantization at compile time—a hard constraint that would reject all three failing models outright, as its compiler cannot ingest mixed-precision graphs with unsupported dynamic operations. The Apple Neural Engine (ANE) operates natively at FP16 and supports INT8 through CoreML's post-training quantization pipeline; however, CoreML's GNN operator coverage is similarly restricted, with `Gather`, `Scatter`, and dynamic indexing operations falling back to the CPU or GPU co-processor. Qualcomm's Hexagon NPU, while more flexible with its 8-bit vector extensions and shared memory architecture, also reports degraded GNN throughput under quantization due to the same memory-bound scaling behavior observed here—sparse aggregation throughput is gated by LPDDR bandwidth rather than compute throughput, making precision reduction ineffective [9].

What distinguishes the Meteor Lake NPU is the nature of the failure mode: rather than providing degraded but functional INT8 execution (as the Hexagon does), or rejecting the model at compile time (as the Edge TPU would), the OpenVINO toolchain silently falls back to CPU for unsupported quantized operators, producing misleading latency numbers that appear to show NPU execution but actually reflect CPU performance. This silent-fallback behavior—observed for MobileNetV2 INT8, ResNet50 INT8, and BERT-Tiny INT8—is a toolchain maturity issue as much as a hardware limitation. As NPU compilers evolve toward more explicit mixed-precision support and per-operator quantization control, the fraction of GNN operators that can benefit from reduced precision should increase, but the fundamental memory-bandwidth bottleneck will persist for any architecture that does not incorporate sparse-dataflow acceleration.

B. Platform Maturity Assessment

The Meteor Lake NPU, as of OpenVINO 2024.1, is best characterized as a vision-first accelerator. Dense convolutional models achieve 4.5–11.4× speedups over CPU (MobileNetV2: 4.5×, ResNet50: 8.0×, ViT-Tiny: 11.4×) and can approach or exceed iGPU performance at lower TDP. The SoCWatch power data confirm that for GNN workloads, CPU and GPU backends operate at comparable power envelopes (9.1–12.5 W package power, measured on GCN and MPNN), with GPU drawing modestly higher power (+7.3% for GCN) while delivering similar latency. GNN support remains at an early stage: operator coverage is incomplete (MPNN falls back entirely to CPU), quantization is unreliable for non-CNN architectures, and the memory subsystem is not designed for sparse, irregular access patterns.

The iGPU's consistent latency advantage over both CPU and NPU for GNN workloads warrants analytical explanation. Three architectural factors contribute. First, the Xe-LPG iGPU in Meteor Lake is backed by a dedicated 128-bit LPDDR5x memory controller with a peak theoretical bandwidth of approximately 120 GB/s—substantially higher than the NPU's shared SoC fabric bandwidth, which must also serve the media engine and display controller. Since GNN execution is memory-bound (Section IV-E, Fig. 6), raw bandwidth directly translates to lower latency. Second, the iGPU's L2 cache (approximately 4–8 MB on the Xe-LPG tile) is an order of magnitude larger than the NPU's explicitly managed SRAM, allowing more of the irregularly accessed feature data to be served from on-die memory rather than DRAM. Third, the OpenVINO GPU plugin applies more aggressive kernel fusion to sparse operations than the NPU plugin, as evidenced by the GPU's higher optimization speedup (1.1–1.25× vs. 1.0–1.05× for NPU, Section IV-E). This fusion advantage stems from the GPU compiler's longer development history and more mature support for dynamic index patterns; the NPU plugin, being relatively new, lacks comparable fusion passes for Gather/Scatter-dominated subgraphs. These three factors—bandwidth, cache capacity, and compiler maturity—collectively explain why the iGPU, despite not being designed for sparse workloads, outperforms the NPU on GNN inference. These hardware-level constraints align with algorithmic evidence that GNNs are not always the optimal choice for sparse graph problems [27], where well-engineered feature-based models can achieve competitive accuracy with lower computational cost.

C. Comparison with Prior Work

Custom GNN accelerators such as HyGCN [2] ($10^3\times$ over CPU), EnGN [1] ($1.8 \times 10^3\times$ over CPU), and GRIP [3] (17× over CPU) demonstrate the potential of hardware-native sparse engines and dedicated memory hierarchies for graph workloads, but rely on custom silicon unavailable in commodity edge platforms. The Meteor Lake NPU provides negligible acceleration for GNNs (0.9–1.2× over CPU). This gap reflects fundamentally different design goals: ASIC accelerators optimize for graph-specific dataflows, while the consumer NPU targets general-purpose dense inference. These findings are consistent with recent analyses of memory bottlenecks in edge AI accelerators [9], [28].

D. Limitations

Several factors constrain the generality and statistical rigor of these findings.

Single-platform scope. All measurements were collected on a single Core Ultra 5 125H processor (Meteor Lake, 4P+8E+2LPE cores, 7 Xe-cores iGPU, one NPU tile). The Core Ultra family spans multiple SKUs with varying core counts, iGPU configurations, and memory controller configurations (LPDDR5x vs. DDR5), all of which affect memory bandwidth and thus the memory-bound GNN latency observed here. Results should not be extrapolated to other Meteor Lake SKUs or to subsequent architectures (Lunar Lake, Arrow Lake) without validation.

Statistical reporting. Reported latencies are arithmetic means over 100 iterations × 3 independent repeats; per-iteration standard deviations are visualized as error bars in Fig. 1. Bootstrap 95% confidence intervals are computed at the per-configuration level (available in the released scalability matrices) and show typical widths of 0.1–0.5 ms for most NPU configurations. However, formal paired hypothesis tests comparing device pairs are not provided, and the confidence

intervals are not propagated into aggregate cross-dataset averages. Small differences between backends—such as the $\sim$6% CPU–NPU latency gap for most GNNs—are not assessed for statistical significance. Given the run-to-run variability inherent in a shared-resource laptop environment (thermal throttling, OS scheduler preemption, DVFS transitions), some of the reported device rankings may not be statistically distinguishable. Future work should incorporate paired t-tests or non-parametric alternatives where normality assumptions are violated, and report effect sizes alongside point estimates.

Power measurement coverage. The SoCWatch power data (Table IV) cover only two of 14 models (GCN and MPNN) and two of three backends (CPU and GPU). The "9.1–12.5 W" power range cited throughout this paper is based on these four configurations and should not be interpreted as a universal characterization of all GNN workloads on this platform. NPU-specific power isolation was not available through the PMT interface: we examined the SoCWatch PMT counter definitions for Meteor Lake and found that the available counters include NPU memory bandwidth (VPU_MEMORY_BW), NPU temperature, and NPU D-state residency counters, but no NPU power or energy rail counter is defined. Consequently, the SoCWatch CLI produced only raw ETW trace files (no CSV summary) for NPU-targeted sessions. Extending energy analysis to all 14 models and to the NPU backend requires either a future SoCWatch release with NPU power rail support or external power instrumentation. Future work should address this gap through dedicated hardware instrumentation—such as current-sense amplifiers or precision shunt resistors on the NPU power rail—to obtain isolated, inference-specific power measurements independent of the SoCWatch PMT interface.

Energy-per-inference estimation. The per-inference energy values in Table IV are computed as Avg Power $\times$ Mean Latency. This linear approximation assumes stationary package power across the 30 s SoCWatch collection window and does not isolate inference-specific energy from background system activity (OS services, display refresh, DRAM self-refresh). In a non-real-time laptop environment, background processes can inject non-trivial power variance; the reported values should be treated as upper-bound estimates.

Compiler and input constraints. The benchmark uses fixed-shape inputs consistent with ONNX static-graph requirements; dynamic-shape execution may introduce additional overhead not captured here. Three architectures (GAT, GATv2, EfficientNet-B0) failed INT8 compilation entirely; potential mitigation strategies such as manual per-operator quantization assignment or partial (mixed-precision) quantization were identified but not pursued, as the ONNX Runtime dynamic quantization API does not expose fine-grained per-operator control—leaving this as an open problem for future toolchain releases.

Preprint dependency. Several comparisons in this paper draw on Bi et al.'s roofline-based benchmarking framework [9], which, at the time of writing, is available only as an arXiv preprint and has not undergone peer review. The framework's quantitative claims about edge accelerator memory-compute balance should be interpreted with appropriate caution until validated by independent reproduction.

Finally, a single toolchain version (OpenVINO 2024.1, ONNX Runtime 1.18) was evaluated; OpenVINO is a rapidly evolving library, and subsequent driver and plugin updates may alter the performance characteristics reported. The results presented here represent the current state of software maturity; as operator coverage expands in future releases, the landscape for NPU-based GNN inference is expected to shift.

VI. Conclusion

This study set out to answer three questions about GNN inference on the Meteor Lake NPU: expected latency, INT8 quantization effectiveness, and toolchain limitations. The answers are nuanced. For dense vision models, the NPU is unequivocally strong—ResNet50 and ViT-Tiny achieve 8–11$\times$ CPU speedups at the NPU's lower TDP. For GNNs, the picture is fundamentally different: SpMM-dominated execution is memory-bound on all three backends, and the NPU offers no meaningful latency advantage over the CPU for most graph models. Where the NPU does differentiate itself is in energy-proportional computing for dense workloads; for sparse workloads, package-level power measurements for two representative GNNs (GCN, MPNN) confirm that the CPU and GPU already operate within a narrow 9–12.5 W band, leaving little room for NPU energy savings absent latency improvements.

The INT8 story is perhaps the most instructive result. Quantization—widely marketed as a key NPU value proposition—fails to deliver for GNNs not because of any single hardware defect, but because of a systemic mismatch: the NPU's quantization toolchain assumes dense, regular computation graphs with static memory access patterns. GNNs violate every term of this contract. The result is not graceful degradation but a spectrum of failures: silent CPU fallback (vision models), outright compilation rejection (GAT, GATv2, EfficientNet-B0), and performance regression even when compilation succeeds (SGC: 2.2$\times$ slower). This pattern is consistent with the broader edge-AI landscape—Edge TPU, ANE, and Hexagon all exhibit similar GNN quantization gaps—suggesting that the problem is architectural rather than vendor-specific.

For practitioners, the practical guidance is straightforward. On current-generation Meteor Lake hardware with OpenVINO 2024.1, deploy dense vision models to the NPU at FP32 for optimal latency and energy efficiency. For GNN inference, use the iGPU as the default backend; it consistently delivers the lowest latency across all GNN architectures tested. Avoid INT8 quantization for GNNs on the NPU unless the specific model has been verified to compile and execute correctly (GraphTransformer and GCN are the only models in this study that passed both checks). Do not rely on reported NPU INT8 latency for vision models without verifying CPU fallback ratios—the toolchain's silent fallback behavior can produce misleading results.

Looking forward, four developments would meaningfully change this assessment. First, the OpenVINO NPU plugin must expose explicit mixed-precision control, allowing practitioners to quantize only the operators that benefit from reduced precision while leaving sparse-indexing operations at FP32. Second, sparse-dataflow support in the NPU memory subsystem—even a lightweight gather engine or strided DMA—would address the dominant memory bottleneck that makes all three backends perform similarly for GNNs. Third, as NPU compiler maturity improves and operator coverage expands, the current compilation failures (GAT, GATv2, EfficientNet-B0) should resolve; this study provides a concrete operator-level checklist for tracking that progress. Fourth, dedicated hardware instrumentation—precision shunt resistors, current-sense amplifiers, or oscilloscope-based rail profiling on the NPU power delivery network—would enable isolated power measurements that the SoCWatch PMT interface cannot provide, allowing definitive NPU energy efficiency characterization. The benchmarking suite, all 14 ONNX models, and the automated analysis pipeline are publicly released at https://yusufarbc.github.io/intel-npu-gnn-benchmarking/ to enable exactly this kind of longitudinal tracking across future platform and toolchain revisions.

REFERENCES

[1] S. Liang, Y. Wang, C. Liu, L. He, H. LI, D. Xu, and X. Li, "Engn: A high-throughput and energy-efficient accelerator for large graph neural networks," *IEEE Transactions on Computers*, vol. 70, no. 9, pp. 1511–1525, Sep. 2021.

[2] M. Yan, L. Deng, X. Hu, L. Liang, Y. Feng, X. Ye, Z. Zhang, D. Fan, and Y. Xie, "Hygcn: A gcn accelerator with hybrid architecture," in *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb 2020, pp. 15–29.

[3] K. Kiningham, P. Levis, and C. Ré, "Grip: A graph neural network accelerator architecture," *IEEE Transactions on Computers*, vol. 72, no. 4, pp. 914–925, April 2023.

[4] T. Lam, "Meteor lake npu architecture deep dive," Intel Community Blog, 2024, available: https://community.intel.com.

[5] I. Corporation, "Intel core ultra processor: Meteor lake architecture," Hot Chips 2023 Presentation, August 2023, available: https://www.intel.com.

[6] ——, "Intel ai boost: Npu architecture for core ultra processors," Intel Architecture Documentation, 2023, available: https://www.intel.com/content/www/us/en/products/docs/processors/core-ultra/ai-boost-npu.html.

[7] ——, "Openvino npu plugin documentation," OpenVINO Documentation, 2024, available: https://docs.openvino.ai.

[8] V. J. Reddi *et al.*, "Mlperf inference benchmark," *arXiv preprint arXiv:1911.02549*, 2020.

[9] Z. Bi *et al.*, "Roofline-based benchmarking framework for on-device ai accelerators," *arXiv preprint*, 2026.

[10] T. N. Kipf and M. Welling, "Semi-supervised classification with graph convolutional networks," in *International Conference on Learning Representations*, 2017. [Online]. Available: https://openreview.net/forum?id=SJU4ayYgl

[11] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations*, 2018. [Online]. Available: https://openreview.net/forum?id=rJXMpikCZ

[12] S. Brody, U. Alon, and E. Yahav, "How attentive are graph attention networks?" in *International Conference on Learning Representations*, 2022. [Online]. Available: https://openreview.net/forum?id=F72ximsx7C1

[13] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How powerful are graph neural networks?" in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=ryGs6iA5Km

[14] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive representation learning on large graphs," in *Proceedings of the 31st International Conference on Neural Information Processing Systems*, ser. NIPS'17. Red Hook, NY, USA: Curran Associates Inc., 2017, p. 1025–1035.

[15] F. Wu, A. Souza, T. Zhang, C. Fifty, T. Yu, and K. Weinberger, "Simplifying graph convolutional networks," in *Proceedings of the 36th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, K. Chaudhuri and R. Salakhutdinov, Eds., vol. 97. PMLR, 09–15 Jun 2019, pp. 6861–6871. [Online]. Available: https://proceedings.mlr.press/v97/wu19e.html

[16] J. Gasteiger, A. Bojchevski, and S. Günnemann, "Predict then propagate: Graph neural networks meet personalized pagerank," in *International Conference on Learning Representations*, 2019. [Online]. Available: https://openreview.net/forum?id=H1gL-2A9Ym

[17] Y. Shi, Z. Huang, S. Feng, H. Zhong, W. Wang, and Y. Sun, "Masked label prediction: Unified message passing model for semi-supervised classification," in *Proceedings of the Thirtieth International Joint Conference on Artificial Intelligence*, 2021, pp. 1548–1554.

[18] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural message passing for quantum chemistry," in *Proceedings of the 34th International Conference on Machine Learning*, ser. Proceedings of Machine Learning Research, D. Precup and Y. W. Teh, Eds., vol. 70. PMLR, 06–11 Aug 2017, pp. 1263–1272. [Online]. Available: https://proceedings.mlr.press/v70/gilmer17a.html

[19] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," pp. 770–778, 2016.

[20] M. Sandler, A. Howard, M. Zhu, A. Zhmoginov, and L.-C. Chen, "Mobilenetv2: Inverted residuals and linear bottlenecks," in *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018, pp. 4510–4520.

[21] M. Tan and Q. V. Le, "Efficientnet: Rethinking model scaling for convolutional neural networks," in *Proceedings of the 36th International Conference on Machine Learning*, 2019, pp. 6105–6114.

[22] A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, J. Uszkoreit, and N. Houlsby, "An image is worth 16x16 words: Transformers for image recognition at scale," in *International Conference on Learning Representations*, 2021. [Online]. Available: https://openreview.net/forum?id=YicbFdNTTy

[23] I. Turc, M.-W. Chang, K. Lee, and K. Toutanova, "Well-read students learn better: On the importance of pre-training compact models," *arXiv preprint arXiv:1908.08962*, 2019.

[24] W. Hu, M. Fey, M. Zitnik, Y. Dong, H. Ren, B. Liu, M. Catasta, and J. Leskovec, "Open graph benchmark: Datasets for machine learning on graphs," *Advances in Neural Information Processing Systems*, vol. 33, pp. 22 118–22 133, 2020. [Online]. Available: https://ogb.stanford.edu

[25] W. Niu, J. Guan, Y. Wang, G. Agrawal, and B. Ren, "Dnnfusion: accelerating deep neural networks execution with advanced operator fusion," in *Proceedings of the 42nd ACM SIGPLAN International Conference on Programming Language Design and Implementation*, ser. PLDI 2021. New York, NY, USA: Association for Computing Machinery, 2021, p. 883–898. [Online]. Available: https://doi.org/10.1145/3453483.3454083

[26] I. Corporation, "Intel soc watch for windows os: User guide," Intel Software Documentation, 2026, version 2026.2. Available: https://www.intel.com/content/www/us/en/developer/topic-technology/open/socwatch/overview.html.

[27] M. Bayraktar *et al.*, "Are gnns always necessary? a comparative study of feature-based and graph-based models for sparse graphs," in *Proceedings of the AAAI Conference on Artificial Intelligence*, 2026.

[28] R. Singh and S. S. Gill, "Edge ai: A survey," *Internet of Things and Cyber-Physical Systems*, vol. 3, pp. 71–92, 2023. [Online]. Available: https://www.sciencedirect.com/science/article/pii/S2667345223000196